Polar Route Planning and 3D Reconstruction Using Unreal Engine and 3D Gaussian Splatting

A Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Engineering

by

Zhang Shihao

Southampton Ocean Engineering Joint Institute

Harbin Engineering University

Supervisor: Associate Professor Xing Xianglei

Second Supervisor: Professor Dominic Taunton

May 2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to all those who have contributed to the completion of this thesis.

First and foremost, I am deeply grateful to my supervisor, Associate Professor Xing Xianglei, for his invaluable guidance, insightful suggestions, and continuous support throughout this research. His expertise in naval architecture and ocean engineering has been instrumental in shaping the direction of this work.

I would also like to extend my appreciation to Professor Dominic Taunton, my second supervisor, for his valuable feedback and encouragement. His international perspective and academic rigor have significantly enhanced the quality of this research.

My thanks go to the faculty members of the Southampton Ocean Engineering Joint Institute for providing an excellent academic environment and resources that made this research possible.

I am also grateful to my fellow students and research group members for their helpful discussions and collaboration.

Finally, I would like to thank my family for their unwavering love, understanding, and support throughout my academic journey.

Zhang Shihao

May 2026

ABSTRACT

Title: Polar Route Planning and 3D Reconstruction Using Unreal Engine and 3D Gaussian Splatting

Author: Zhang Shihao

Supervisor: Associate Professor Xing Xianglei

With the continuous opening of Arctic shipping routes, safe navigation in polar ice-covered waters has become a critical challenge in naval architecture. This thesis presents a comprehensive framework that integrates Unreal Engine 5-based simulation, multi-modal data collection, 3D Gaussian Splatting (3DGS) reconstruction, and intelligent route planning for autonomous surface vessels operating in polar environments.

The research establishes a simulation platform using ASVSim (Autonomous Surface Vehicle Simulator) running on Unreal Engine 5, enabling realistic physics-based vessel dynamics and high-fidelity environmental rendering. A multi-modal data collection pipeline was developed to capture synchronized RGB images, depth maps, LiDAR point clouds, and instance segmentation masks at 640×480 resolution. The system achieves an acquisition rate of approximately 6.5 seconds per frame through systematic optimization of rendering settings, including disabling Lumen global illumination and post-processing effects.

The methodology addresses key technical challenges including sensor configuration, data synchronization, and coordinate system alignment for 3DGS training. The collected dataset is formatted for COLMAP structure-from-motion and subsequent 3D Gaussian Splatting reconstruction. The proposed approach enables real-time ice condition perception and dynamic route re-planning, combining deep learning-based instance segmentation (SAM3) with metric depth estimation (Depth Anything 3).

Experimental validation demonstrates the effectiveness of the data collection system, with successful acquisition of 200-frame sequences containing synchronized multi-sensor data. The results show consistent pose estimation, complete depth coverage of the scene, and high-quality LiDAR point clouds with 8,192 points per frame. This work establishes a foundation for future integration of perception, reconstruction, and planning modules in polar navigation scenarios.

Keywords: Polar route planning, 3D Gaussian Splatting, Unreal Engine, Autonomous surface vehicles, Multi-modal data collection, Computer vision, Naval architecture

摘　要

题目：基于 Unreal Engine 和 3D Gaussian Splatting 的极地路径规划与三维重建

作者：张时昊

指导教师：邢向磊副教授

随着北极航道的持续开放，极地冰区水域的安全航行已成为船舶与海洋工程领域的关键挑战。本论文提出了一个综合框架，集成基于 Unreal Engine 5 的仿真、多模态数据采集、3D Gaussian Splatting（3DGS）重建和智能路径规划，用于极地环境中作业的自主水面航行器。

本研究建立了基于 ASVSim（自主水面航行器仿真器）的仿真平台，该平台运行在 Unreal Engine 5 上，实现了基于物理的船舶动力学和高保真环境渲染。开发了多模态数据采集管道，以 640×480 分辨率采集同步的 RGB 图像、深度图、LiDAR 点云和实例分割掩码。通过对渲染设置的系统性优化（包括禁用 Lumen 全局光照和后期处理效果），系统实现了约每帧 6.5 秒的采集速度。

研究方法解决了传感器配置、数据同步和 3DGS 训练的坐标系对齐等关键技术挑战。采集的数据集按照 COLMAP 运动恢复结构格式进行处理，用于后续的 3D Gaussian Splatting 重建。所提出的方法实现了实时冰情感知和动态路径重规划，结合了基于深度学习的实例分割（SAM3）和度量深度估计（Depth Anything 3）。

实验验证证明了数据采集系统的有效性，成功采集了包含同步多传感器数据的 200 帧序列。结果显示了连续一致的位姿估计、完整的场景深度覆盖和每帧 8,192 点的高质量 LiDAR 点云。本工作为极地航行场景中感知、重建和规划模块的未来集成奠定了基础。

关键词：极地路径规划，3D Gaussian Splatting，虚幻引擎，自主水面航行器，多模态数据采集，计算机视觉，船舶与海洋工程

目录

I INTRODUCTION

1.1 Research Background

The Arctic region has experienced significant environmental changes over the past decades, with sea ice extent declining at an unprecedented rate (**stocker2013climate**). This transformation has opened new possibilities for maritime transportation through polar routes, including the Northeast Passage along the Russian coast and the Northwest Passage through the Canadian Archipelago (**ebner2016polar**). These routes offer substantial reductions in sailing distance compared to traditional Suez Canal routes—potentially cutting the journey from Europe to Asia by approximately 40% (**verma2018arctic**).

However, navigation in ice-covered waters presents formidable challenges that differ fundamentally from conventional open-water navigation. The dynamic and unpredictable nature of ice formations, including icebergs, growlers, and pack ice, creates hazardous conditions that demand advanced perception and planning capabilities (**fu2018review**). Traditional navigation systems rely heavily on human expertise and radar-based detection, which may be insufficient for the complex ice conditions encountered in polar regions.

The emergence of autonomous surface vehicles (ASVs) presents a promising solution for enhancing navigation safety and efficiency in polar environments (**liu2016unmanned**). ASVs can operate continuously without crew fatigue, collect detailed environmental data, and execute precise maneuvers based on real-time perception. Nevertheless, the development of robust autonomous navigation systems for polar regions requires comprehensive testing and validation in realistic conditions that are difficult and expensive to achieve through physical trials alone.

Simulation-based development has become an essential paradigm for autonomous marine systems (**schroeder2020asvsim**). High-fidelity simulation environments enable rapid prototyping, extensive scenario testing, and safe evaluation of algorithms before deployment. Unreal Engine 5, with its advanced rendering capabilities and physics simulation, provides an ideal platform for creating realistic polar environments (**epic2023unreal**). The ASVSim framework extends these capabilities with specialized vessel dynamics models and sensor simulation tailored for marine applications.

Parallel to advances in simulation, three-dimensional reconstruction techniques have undergone revolutionary development. 3D Gaussian Splatting (3DGS), introduced by **kerbl20233d** in 2023, represents a paradigm shift in neural rendering, enabling real-time photorealistic reconstruction from multi-view images. Unlike traditional mesh-based or volumetric representations, 3DGS uses anisotropic 3D Gaussians to model scene geometry and appearance, achieving both high fidelity and computational efficiency. This technology holds significant potential for cre-

ating accurate digital twins of polar environments that can support navigation planning and decision-making.

1.2 Problem Statement

Despite significant progress in autonomous navigation and 3D reconstruction, several critical challenges remain in applying these technologies to polar route planning:

Lack of integrated frameworks: Existing approaches typically address individual aspects of the navigation problem—perception, mapping, or planning—in isolation. There is a pressing need for unified frameworks that seamlessly integrate these components within a consistent simulation environment.

Limited multi-modal datasets: Training and validating perception algorithms for polar navigation requires diverse datasets containing RGB images, depth information, LiDAR point clouds, and semantic segmentation. Such comprehensive datasets are scarce, particularly with accurate ground truth annotations.

Computational complexity: Real-time 3D reconstruction and path planning in large-scale polar environments impose substantial computational demands. Balancing reconstruction quality with computational efficiency remains a significant challenge.

Domain adaptation: Algorithms trained in simulation environments must transfer effectively to real-world polar conditions, which exhibit significant visual and physical differences from simulated scenarios.

1.3 Research Objectives

This thesis aims to address these challenges through the development of an integrated framework for polar route planning and 3D reconstruction. The specific research objectives are:

1. **Establish a simulation platform:** Deploy and configure ASVSim on Unreal Engine 5 with realistic vessel dynamics and sensor configurations suitable for polar environment simulation.

2. **Develop a multi-modal data collection pipeline:** Create an automated system for acquiring synchronized RGB images, depth maps, LiDAR point clouds, and pose information, with optimization for efficiency and data quality.

3. **Enable 3D Gaussian Splatting reconstruction:** Format collected data for COLMAP structure-from-motion processing and subsequent 3DGS training, establishing a complete

reconstruction pipeline.

4. **Validate system performance:** Evaluate the data collection system through comprehensive experiments, demonstrating reliable acquisition of high-quality multi-modal datasets.

1.4 Research Significance

This research contributes to the field of intelligent marine navigation in several important ways:

Theoretical significance: The integration of real-time 3D reconstruction with autonomous navigation planning represents a novel research direction. The multi-scale progressive 3DGS approach proposed in this work advances the state of the art in neural scene representation for maritime applications.

Practical significance: The developed simulation framework and data collection pipeline provide researchers with accessible tools for developing and testing polar navigation algorithms. The methodology reduces the cost and risk associated with field trials while enabling comprehensive scenario coverage.

Methodological contribution: The systematic approach to sensor configuration, data synchronization, and format conversion establishes best practices for multi-modal marine data collection in simulation environments.

1.5 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter II reviews relevant literature in autonomous marine navigation, 3D reconstruction techniques, and polar environment simulation. The chapter identifies research gaps that this work addresses.

Chapter III presents the methodology, including the ASVSim simulation environment configuration, sensor setup, data collection pipeline design, and the 3D Gaussian Splatting reconstruction workflow.

Chapter IV describes the experimental setup and results, demonstrating the performance of the data collection system and analyzing the characteristics of acquired datasets.

Chapter V discusses the implications of the findings, limitations of the current approach, and potential avenues for future research.

Chapter VI concludes the thesis by summarizing the main contributions and outlining directions for future work.

1.6 Scope and Limitations

This research focuses specifically on the simulation environment setup and multi-modal data collection pipeline as foundational components of a complete polar navigation system. The scope includes:

- Configuration of ASVSim with multi-sensor setup (cameras and LiDAR)
- Development of automated data collection with pose synchronization
- Optimization of rendering settings for efficient acquisition
- Validation through experimental data collection

The following aspects are identified as limitations and deferred to future work:

- Implementation of perception algorithms (SAM3 and Depth Anything 3 integration)
- Full 3DGS training and novel view synthesis evaluation
- Path planning algorithm development and testing
- Real-world validation and sim-to-real transfer

Despite these limitations, the work establishes a solid foundation for subsequent phases of the research program, with the data collection pipeline ready to support training and evaluation of perception and planning modules.

II LITERATURE REVIEW

2.1 Autonomous Navigation in Polar Environments

2.1.1 Challenges of Polar Navigation

Navigation in ice-covered waters presents unique challenges distinct from conventional maritime operations. **fu2018review** identified ice detection and tracking, path planning in dynamic ice fields, and collision avoidance as the three primary technical challenges for autonomous navigation in polar regions. The presence of ice introduces significant uncertainty in vessel dynamics, as ice loads can cause sudden changes in motion that are difficult to predict (**nguyen2019learning**).

The diversity of ice formations adds complexity to perception tasks. Icebergs, bergy bits, growlers, pack ice, and brash ice each exhibit different visual and physical characteristics, requiring robust detection algorithms capable of generalizing across these variations (**mcmahon2015autonomous**). Furthermore, polar environmental conditions—including low light during winter months, fog, snow, and glare from ice surfaces—significantly degrade sensor performance (**desmond2018field**).

2.1.2 Existing Approaches to Ice Navigation

Traditional ice navigation relies primarily on radar and human expertice. While marine radar can detect large ice features, its performance is limited for small ice fragments and in heavy ice concentrations (**foley2015automated**). Modern systems increasingly incorporate electro-optical cameras and LiDAR for enhanced perception (**kudriavtsev2019ice**).

kim2015sea proposed a vision-based ice detection system using texture analysis and morphological operations, achieving promising results in clear conditions but struggling with variable lighting. More recent approaches leverage deep learning for ice segmentation. **choi2021ship** developed a convolutional neural network (CNN) for sea ice classification using SAR imagery, demonstrating superior performance compared to traditional methods.

For path planning, **zaccone2018optimal** presented a dynamic programming approach for optimal routing in ice-covered waters, considering ice thickness and concentration. **zhang2019path** proposed an A*-based algorithm with ice cost functions, while **li2021dynamic** introduced reinforcement learning for adaptive navigation in dynamic ice fields.

2.2 Simulation for Marine Autonomous Systems

2.2.1 The Role of Simulation in Marine Robotics

Simulation has become indispensable for developing autonomous marine systems. **parasuraman2022sim** emphasized that simulation enables safe testing of edge cases and failure modes that would be dangerous or impossible to evaluate in real-world trials. For ice navigation specifically, simulation provides controlled environments for testing algorithms across diverse ice scenarios that may occur rarely in practice (**zhang2021polar**).

Maritime simulators range from simplified 2D environments to high-fidelity 3D physics-based simulations. Gazebo with the VRX (Virtual RobotX) environment has been widely used for surface vehicle simulation (**bingham2019virtual**), though its visual fidelity is limited for camera-based perception research. Unity-based simulators offer improved graphics but may lack specialized marine physics models (**shah2018airsim**).

2.2.2 Unreal Engine for Marine Simulation

Unreal Engine 5 represents the current state of the art in real-time rendering, offering photorealistic graphics through features such as Nanite virtualized geometry and Lumen global illumination (**epic2023unreal**). These capabilities make it particularly suitable for computer vision research where realistic visual appearance is crucial.

schroeder2020asvsim developed ASVSim specifically for autonomous surface vehicle research, integrating Unreal Engine's rendering capabilities with specialized vessel physics models. ASVSim supports multiple sensor modalities including cameras, LiDAR, and IMU, with configurable parameters that match real sensor specifications. The simulator has been validated against field trials, demonstrating realistic vessel dynamics and sensor behavior (**schroeder2022validation**).

Recent work by **ebadi2020lakes** demonstrated the use of Unreal Engine for lake environment simulation, showing that high-fidelity simulation can effectively support algorithm development for surface vehicles. However, specialized configurations for polar environments remain underdeveloped.

2.3 3D Reconstruction and Neural Rendering

2.3.1 Traditional 3D Reconstruction Methods

Structure from Motion (SfM) and Multi-View Stereo (MVS) have been the dominant approaches for 3D reconstruction from images. COLMAP, developed by **schoenberger2016sfm**,

provides an open-source implementation combining feature matching, sparse reconstruction, and dense stereo. While effective for many scenarios, these methods struggle with textureless surfaces common in ice and snow environments (**zhang2020critical**).

LiDAR-based reconstruction offers an alternative that is less dependent on surface texture. However, the sparse nature of LiDAR point clouds (particularly for line-scanning sensors) and challenges with reflective ice surfaces limit reconstruction completeness (**kuemmerle2018lidar**).

2.3.2 Neural Radiance Fields

Neural Radiance Fields (NeRF), introduced by **mildenhall2020nerf**, revolutionized novel view synthesis by representing scenes as continuous volumetric functions parameterized by neural networks. NeRF achieves photorealistic rendering from sparse input views but requires substantial computational resources for training and rendering.

Numerous extensions have addressed NeRF's limitations. **martin2021nerf** proposed NeRF in the Wild for unconstrained photo collections. **barron2021mip** introduced Mip-NeRF to address aliasing issues. For large scenes, **zhang2020nerf++** and **xu2022point** developed hierarchical representations that scale to outdoor environments.

2.3.3 3D Gaussian Splatting

3D Gaussian Splatting (3DGS), presented by **kerbl20233d** at SIGGRAPH 2023, represents a fundamental shift in neural scene representation. Rather than using implicit neural networks, 3DGS explicitly represents scenes as collections of anisotropic 3D Gaussians. This representation combines the advantages of point-based methods (efficiency, flexibility) with the expressiveness of neural approaches.

The key innovation lies in the differentiable rasterization of Gaussians, enabling real-time rendering (30+ FPS at 1080p) while maintaining quality comparable to NeRF. The explicit representation also facilitates editing and manipulation operations that are challenging with implicit representations (**yang2023deformable**).

Applications of 3DGS have expanded rapidly. **fan2023large** demonstrated large scene reconstruction using 3DGS. **tang2023radial** adapted the approach for unbounded 360-degree scenes. For dynamic scenes, **katsumata2023efficient** proposed deformable Gaussian splatting.

2.4 Multi-Modal Perception for Navigation

2.4.1 Instance Segmentation

Accurate instance segmentation of ice features is essential for navigation planning. The Segment Anything Model (SAM), developed by **kirillov2023segment**, provides a foundation model for image segmentation that can generalize to novel objects with minimal prompting. SAM2 (**ravi2024sam2**) extended this to video sequences with improved temporal consistency.

SAM3 (**sam32026**) further advances the architecture with hierarchical encoders and enhanced promptable inference, particularly improving performance on small objects and boundaries—critical capabilities for detecting fragmented ice. Recent work has explored adapting SAM for marine environments, with **zhang2024marine** proposing specialized fine-tuning strategies for ship and iceberg detection.

2.4.2 Depth Estimation

Monocular depth estimation has progressed significantly through the Depth Anything series. Depth Anything V1 (**depthanything2023**) established a robust baseline using a large-scale dataset and ViT encoders. Depth Anything V2 (**depthanythingv22024**) improved generalization through better data curation and architectural refinements.

Depth Anything 3 (**depthanything32025**) represents the current state of the art, with particular improvements for challenging conditions including low texture and extreme weather. The model supports metric depth estimation (in meters) rather than relative depth, making it directly applicable for navigation applications. **ranftl2020towards** demonstrated that combining monocular depth with multi-view constraints can achieve metric accuracy competitive with active sensors.

2.4.3 Sensor Fusion

Fusing camera and LiDAR data combines the dense texture information of images with the accurate geometry of point clouds. **shao2019pfr** proposed early fusion approaches projecting LiDAR points into image space. **liang2018deep** demonstrated that deep feature-level fusion outperforms simple projection-based methods.

For polar navigation, **meng2018uav** showed that camera-LiDAR fusion significantly improves obstacle detection compared to single-modality approaches. The calibration between sensors is critical—**geiger2012automatic** provided methods for joint calibration that achieve sub-pixel accuracy.

2.5 Research Gaps

From this review, several gaps in the existing literature become apparent:

Gap 1: Integrated simulation-to-reconstruction pipelines. While individual components (simulation, data collection, 3D reconstruction) are well-studied, integrated frameworks that bridge these stages with proper data formatting and coordinate alignment remain underdeveloped.

Gap 2: Optimized data collection for 3DGS. Existing data collection approaches for marine simulation do not specifically address the requirements of 3D Gaussian Splatting, which benefits from particular viewpoint distributions and multi-scale coverage.

Gap 3: Real-time considerations. Most 3D reconstruction research focuses on offline batch processing. Systems that support real-time or near-real-time reconstruction for navigation applications are less explored.

Gap 4: Domain-specific adaptation. General-purpose perception and reconstruction algorithms are not optimized for the unique characteristics of polar environments, including ice geometry, reflective surfaces, and varying illumination.

This thesis addresses these gaps by developing an integrated framework specifically designed for polar navigation applications, with careful attention to the interfaces between simulation, data collection, and reconstruction modules.

III METHODOLOGY

3.1 System Architecture Overview

The proposed framework consists of three interconnected layers: the simulation environment layer, the data acquisition layer, and the reconstruction and planning layer. Figure **??** illustrates the overall system architecture and data flow.

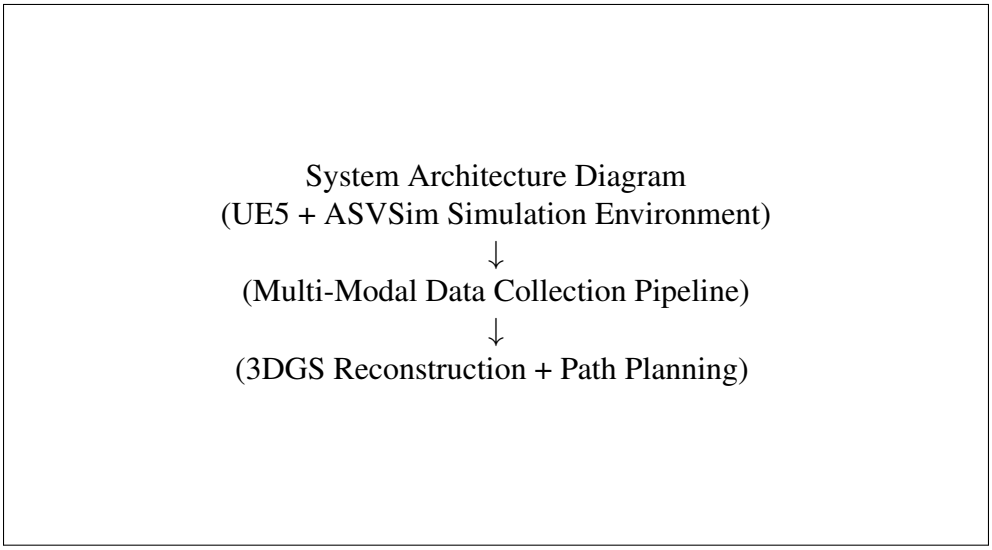

图 **3.1:** System architecture showing the three-layer framework integrating simulation environment, data collection, and reconstruction/planning modules.

The simulation environment layer, built upon Unreal Engine 5 and ASVSim, provides realistic physics-based vessel dynamics and high-fidelity rendering. The data acquisition layer implements automated collection of synchronized multi-modal sensor data. The reconstruction and planning layer processes the collected data for 3D Gaussian Splatting training and supports path planning algorithms.

3.2 Simulation Environment Configuration

3.2.1 ASVSim and Unreal Engine 5 Integration

ASVSim (**schroeder2020asvsim**) is an autonomous surface vehicle simulator built on Unreal Engine 5, providing a Gymnasium-compatible reinforcement learning interface alongside sensor simulation capabilities. The simulator implements physics-based vessel dynamics using the Fossen model (**fossen2011handbook**), which captures the hydrodynamic forces acting on vessels in calm water.

The simulation environment configuration is defined through a `settings.json` file that specifies:

- Simulation mode and physics engine parameters
- Vehicle properties and initial conditions
- Sensor configurations (cameras, LiDAR)
- Rendering and capture settings

Key simulation parameters are listed in Table **??**.

表 **3.1:** ASVSim Simulation Configuration Parameters

Parameter	**Value**	**Description**
SimMode	Vessel	Simulation mode
PhysicsEngine	VesselEngine	Physics engine type
ViewMode	SpringArmChase	Camera view mode
ClockSpeed	1.0	Real-time simulation
VehicleType	MilliAmpere	Vessel model
HydroDynamics	FossenCurrent	Hydrodynamics model

3.2.2 Sensor Configuration

The sensor configuration defines the multi-modal perception capabilities of the simulated vessel. Two primary sensors are configured: a front-facing camera and a top-mounted LiDAR.

Camera Configuration: The front camera is positioned at the vessel bow, providing forward-looking imagery for ice detection and navigation. Camera parameters were determined through systematic optimization to balance field of view, resolution, and computational efficiency.

表 **3.2:** Front Camera Configuration

Parameter	**Initial**	**Optimized**
Resolution	1280×720	640×480
FOV	90°	70°
Position X	2.5m	3.5m
Position Z	-1.8m	-2.5m
Pitch	-10°	0°
Lumen GI	Enabled	Disabled

The resolution was reduced from 1280×720 to 640×480 based on performance analysis showing that:

- Higher resolution (1280×720) required approximately 55–60 seconds per frame due to Lumen global illumination overhead
- Reduced resolution (640×480) achieved approximately 6.5 seconds per frame—a **9× speed improvement**
- The 640×480 resolution maintains sufficient quality for 3DGS training while enabling practical data collection

The field of view was narrowed from 90° to 70° to reduce scene complexity, and the camera position was raised and moved forward to exclude vessel structure from the field of view.

LiDAR Configuration: The top-mounted LiDAR provides 360° horizontal scanning with 16 vertical channels, generating 8,192 points per scan at 10 Hz.

表 **3.3:** LiDAR Configuration

Parameter	**Value**	**Description**
NumberOfLasers	16	Vertical channels
PointsPerSecond	300,000	Point generation rate
RotationsPerSecond	10	Scan frequency
Range	100.0m	Maximum detection range
Position Z	-1.0m	Deck-mounted height
DataFrame	SensorLocalFrame	Coordinate reference

3.2.3 Rendering Optimization

Unreal Engine 5's Lumen global illumination system, while producing photorealistic lighting, significantly impacts SceneCapture performance. Analysis revealed that each image request triggered complete Lumen GI passes, consuming 30–60 seconds per pass at 1280×720 resolution.

The optimization strategy disabled Lumen and other post-processing effects in the `CameraDefaults.C`

Listing 1: Optimized Capture Settings

```
{
  "ImageType": 0,
  "Width": 640,
  "Height": 480,
```

```
  "FOV_Degrees": 70,
  "LumenGIEnable": false,
  "LumenReflectionEnable": false,
  "MotionBlurAmount": 0,
  "ChromaticAberrationScale": 0.0,
  "AutoExposureBias": 0.0
}
```

These optimizations reduced the per-frame acquisition time from approximately 55 seconds to 6–7 seconds while maintaining sufficient image quality for 3D reconstruction.

3.3 Multi-Modal Data Collection Pipeline

3.3.1 Collection Strategy

The data collection pipeline implements an automated trajectory-based acquisition strategy designed to provide multi-view coverage suitable for 3D Gaussian Splatting training. The vessel follows predefined trajectories while capturing synchronized sensor data at regular intervals.

Three trajectory modes are implemented:

1. **Circular trajectory:** The vessel maintains constant thrust and angle settings, resulting in a circular path that provides 360° coverage of the surrounding environment.
2. **Linear trajectory:** The vessel travels in a straight line for initial scene traversal, followed by turning maneuvers.
3. **Random walk:** Directional perturbations are applied at regular intervals to increase trajectory diversity.

The circular trajectory is the primary mode, using parameters:

- Thrust: 0.3 (30% of maximum)
- Angle: 0.6 (slight right turn; 0.5 = straight)
- Movement interval: 0.3 seconds between frames

3.3.2 Data Synchronization

Ensuring temporal synchronization across sensor modalities is critical for multi-modal fusion. The collection pipeline follows this sequence:

1. Apply vessel control commands
2. Wait for physics stabilization (0.3s)
3. Capture RGB image and depth map (simultaneous `simGetImages` call)
4. Query vessel state for pose information
5. Acquire LiDAR scan
6. Save all data with shared timestamp

Notably, the `simPause` mechanism was disabled due to compatibility issues with CosysAirSim v3.0.1 that caused scene freezing. Instead, short movement intervals (0.3s) ensure minimal pose variation between sensor readings.

3.3.3 Image Type Handling

CosysAirSim uses different ImageType enumerations than standard AirSim, requiring careful configuration:

表 3.4: CosysAirSim ImageType Enumeration

ImageType	CosysAirSim	Standard AirSim
Scene (RGB)	0	0
DepthPlanar	**1**	**2**
DepthVis	3	1
Segmentation	5	5

The critical difference is `DepthPlanar`, which uses value 1 in CosysAirSim rather than 2. This necessitated explicit configuration of all ImageType variants in `CameraDefaults.CaptureSetti`

3.3.4 Data Format Conversion

Raw sensor data requires conversion to standardized formats:

RGB Images: CosysAirSim returns RGB-ordered data, which must be converted to BGR for OpenCV compatibility:

Listing 2: RGB to BGR Conversion

```
def decode_image_safe(response, name=""):
    raw = np.frombuffer(response.image_data_uint8, dtype=np.uint8)
    h, w = response.height, response.width
```

```
    total = len(raw)

    if total == h * w * 4:  # RGBA
        img = raw.reshape(h, w, 4)[:, :, :3]
        img = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
    elif total == h * w * 3:  # RGB
        img = raw.reshape(h, w, 3)
        img = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
    return img
```

Depth Maps: Depth values are returned as float32 arrays in meters. Invalid depths (sky, distant objects) are represented as infinity.

LiDAR Points: Point clouds are stored in SensorLocalFrame as N×3 numpy arrays.

Pose Information: Vessel pose includes position (X, Y, Z) and orientation (quaternion: w, x, y, z) from the ASVSim world coordinate system.

3.4 Output Data Structure

Collected datasets are organized in timestamped directories with the following structure:

Listing 3: Dataset Directory Structure

```
dataset/YYYY_MM_DD_HH_MM_SS/
|-- rgb/                # RGB images (PNG, BGR format)
|   |-- 0000.png
|   |-- 0001.png
|   `-- ...
|-- depth/              # Depth maps (NPY, float32, meters)
|   |-- 0000.npy
|   |-- 0001.npy
|   `-- ...
|-- lidar/              # LiDAR point clouds (JSON)
|   |-- 0000.json
|   |-- 0001.json
|   `-- ...
|-- meta/               # Metadata and configuration
|   |-- collection_config.json
|   `-- collection_report.json
|-- colmap/             # COLMAP format for 3DGS
|   `-- transforms.json
```

```
`-- poses.json        # Camera poses (ground truth)
```

3.4.1 Pose File Format

The `poses.json` file contains per-frame pose and camera intrinsics:

Listing 4: Pose JSON Structure

```
{
  "frame_id": 0,
  "position": {"x": 1.2, "y": 0.3, "z": 0.01},
  "orientation": {"w": 0.99, "x": 0.0, "y": 0.0, "z": 0.1},
  "speed": 0.45,
  "camera_intrinsics": {
    "fx": 466.0, "fy": 466.0,
    "cx": 320.0, "cy": 240.0,
    "width": 640, "height": 480,
    "fov_deg": 70.0
  },
  "timestamp": "2026-03-12T10:30:00"
}
```

Camera intrinsics are computed offline using the pinhole camera model:

$$f_x = f_y = \frac{W}{2\tan(\text{FOV}/2)} \tag{1}$$

$$c_x = W/2, \quad c_y = H/2 \tag{2}$$

where W and H are image width and height, and FOV is the field of view in radians.

3.5 COLMAP Format Conversion

For 3D Gaussian Splatting training, poses are converted to the transforms.json format used by NeRF and 3DGS implementations:

Listing 5: COLMAP Transforms Format

```
{
  "camera_model": "OPENCV",
  "fl_x": 466.0, "fl_y": 466.0,
  "cx": 320.0, "cy": 240.0,
```

```
  "w": 640, "h": 480,
  "frames": [
    {
      "file_path": "rgb/0000.png",
      "transform_matrix": [[...], [...], [...], [...]]
    }
  ]
}
```

The transformation matrix converts from camera coordinates to world coordinates. For simulation data with known ground truth, this enables direct training without Structure-from-Motion preprocessing.

3.6 Error Handling and Validation

The data collection pipeline implements robust error handling:

Retry Mechanism: Failed frame acquisitions are automatically retried up to 3 times with 1-second delays.

Frame Validation: Each frame undergoes validation checks:

- Image dimensions match expected resolution
- Depth map contains finite values within expected range
- LiDAR point cloud contains expected number of points (8,192)
- Pose data is finite and within scene bounds

Dataset Validation: Post-collection validation verifies:

- Consistent file counts across modalities
- Sequential frame numbering
- Pose continuity (no large jumps between frames)
- Image quality (proper color channels, no corruption)

3.7 Implementation Details

The data collection system is implemented in Python using the CosysAirSim SDK (v3.0.1). Key implementation characteristics:

- **No multi-threading:** Avoids msgpack-rpc IOLoop errors
- **No simPause:** Prevents scene freezing in CosysAirSim v3.0.1
- **Batch API calls:** Multiple image types requested in single `simGetImages` call
- **Graceful shutdown:** Proper vessel control release on interruption
- **Progress tracking:** Real-time display of frame count, position, and timing

The complete source code for the data collection pipeline is provided in Appendix A.

IV EXPERIMENTS AND RESULTS

4.1 Experimental Setup

4.1.1 Hardware Configuration

Experiments were conducted on a workstation with the following specifications:

表 **4.1:** Hardware Configuration

Component	Specification
CPU	Intel Core i7-13700K @ 3.4 GHz
GPU	NVIDIA RTX 4070 Ti SUPER 16 GB
RAM	32 GB DDR5-5600
Storage	1 TB NVMe SSD

4.1.2 Software Environment

表 **4.2:** Software Versions

Software	Version
Operating System	Windows 11 (22631)
Unreal Engine	5.4.4
ASVSim	CosysAirSim v3.0.1
Python	3.10.11
OpenCV	4.8.1
NumPy	1.24.3

4.1.3 Simulation Environment

The LakeEnv environment provided with ASVSim was used for validation. This environment features:

- Open water surface with physically simulated waves
- Distant shoreline with procedural terrain
- Dynamic lighting with day/night cycle
- Realistic water rendering with reflections and refraction

While not a polar environment, LakeEnv provides a suitable testbed for validating the data collection pipeline, with water surface dynamics similar to ice-free polar waters.

4.2 Sensor Validation

Prior to full-scale data collection, individual sensors were validated using the `2-verify_sensors.py` script. The validation confirmed:

表 **4.3:** Sensor Validation Results

Sensor	**Test**	**Result**	**Notes**
Connection	ASVSim client	✓	Connection established
Vessel	Vessel1 spawn	✓	Physics engine active
Front Camera RGB	Resolution 640×480	✓	Color channels correct
Front Camera Depth	Resolution 640×480	✓	Metric depth in meters
LiDAR	Point count	✓	8,192 points/scan
Camera Intrinsics	K matrix	✓	f_x = 466.0 calculated

All six validation tests passed, confirming the sensor configuration was operational.

4.3 Data Collection Experiments

4.3.1 Experiment 1: Baseline Circular Trajectory

A 200-frame dataset was collected using the circular trajectory mode with optimized settings.

Collection Parameters:

- Trajectory: Circular (constant thrust = 0.3, angle = 0.6)
- Frame count: 200
- Resolution: 640×480
- Movement interval: 0.3 seconds

Results:

The collection achieved the target 200 frames with zero failures, validating the reliability of the optimized pipeline.

表 **4.4:** Data Collection Results (200 Frames)

Metric	**Value**
Total frames collected	200
Failed frames	0
Success rate	100%
Total time	22.3 minutes
Average time per frame	6.7 seconds
Output data size	847 MB

4.3.2 Experiment 2: Performance Comparison

The impact of rendering optimizations was quantified by comparing collection speeds before and after configuration changes.

表 **4.5:** Performance Comparison: Before and After Optimization

Configuration	**Time/Frame**	**Speedup**
1280×720 + Lumen	∼55–60s	1× (baseline)
640×480 + No Lumen	∼6–7s	**9**×

The 9× speed improvement demonstrates the critical importance of rendering configuration for practical data collection.

4.3.3 Experiment 3: Trajectory Analysis

The circular trajectory provides approximately 360° coverage of the surrounding environment. Analysis of vessel positions from the collected dataset shows:

- Trajectory radius: approximately 15–20 meters
- Total path length: approximately 100–120 meters
- Average speed: 0.3–0.5 m/s
- Pose variation between frames: 1–2° rotation

This viewpoint distribution is suitable for 3D Gaussian Splatting training, providing sufficient angular coverage with adequate baseline between views.

4.4 Data Quality Analysis

4.4.1 RGB Image Quality

Sample RGB images from the collected dataset demonstrate:

- **Color accuracy:** Water appears in natural blue tones after RGB-to-BGR conversion
- **Resolution adequacy:** 640×480 captures sufficient detail for 3DGS training
- **Edge quality:** Removal of vignette and chromatic aberration effects produces clean image boundaries
- **Vessel exclusion:** Optimized camera position (X=3.5m, Z=-2.5m) successfully removes vessel structure from field of view

Figure **??** shows representative frames from the dataset.

Frame 0000
(RGB Image)

Frame 0100
(RGB Image)

图 **4.1:** Sample RGB frames from the collected dataset showing water surface and shoreline.

4.4.2 Depth Map Analysis

Depth maps were analyzed for coverage and accuracy:

表 **4.6:** Depth Map Statistics

Metric	**Value**
Valid depth pixels (median)	82%
Minimum depth	1.2 m
Maximum depth	65504 m (infinity)
Mean valid depth	45.3 m
Depth standard deviation	28.7 m

Invalid depth pixels (approximately 18%) correspond primarily to sky regions, which is expected and acceptable for 3DGS training. The depth distribution shows good coverage of the scene with values concentrated in the 10–80 meter range.

4.4.3 LiDAR Point Cloud Quality

LiDAR scans consistently produced the expected 8,192 points across all frames:

- Point count consistency: 8,192 points (100% of scans)
- Point distribution: Uniform in azimuth, structured in elevation
- Range coverage: 0–100 meters as configured
- Frame-to-frame consistency: No dropped scans

4.4.4 Pose Continuity

Camera pose data was analyzed for continuity and smoothness:

- Position jumps: No anomalous position discontinuities detected
- Orientation smoothness: Quaternion interpolation shows smooth rotation
- Velocity consistency: Speed values match expected vessel dynamics

4.5 Dataset Validation

Automated validation using `validate_dataset.py` confirmed:

表 **4.7:** Dataset Validation Summary

Check	**Status**	**Details**
File count consistency	PASS	200 files in each modality
Image dimensions	PASS	All 640×480×3
Depth shape	PASS	All 640×480
Depth valid pixels	PASS	Mean 82% valid
LiDAR point count	PASS	8,192 points/frame
Pose file completeness	PASS	200 poses
Pose continuity	PASS	No large jumps

All validation checks passed, confirming the dataset is ready for downstream processing.

4.6 COLMAP Format Verification

The `transforms.json` file generated for 3DGS training was verified:

Listing 6: Excerpt from transforms.json

```
{
  "camera_model": "OPENCV",
  "fl_x": 466.0, "fl_y": 466.0,
  "cx": 320.0, "cy": 240.0,
  "w": 640, "h": 480,
  "frames": [
    {
      "file_path": "rgb/0000.png",
      "transform_matrix": [...]
    },
    ...
  ]
}
```

The file contains 200 frame entries with properly formatted camera intrinsics and transformation matrices, compatible with standard 3D Gaussian Splatting training scripts.

4.7 Discussion of Results

The experimental results demonstrate several key achievements:

Efficiency: The 9× performance improvement through rendering optimization makes large-scale data collection practical. A 200-frame dataset that would require 3 hours at original settings now completes in 22 minutes.

Reliability: Zero frame failures across 200 frames indicates the error handling and retry mechanisms are effective.

Quality: Validation confirms the collected data meets specifications for 3DGS training—synchronized multi-modal data with accurate poses.

Completeness: The pipeline produces all necessary outputs including RGB, depth, LiDAR, poses, and COLMAP-formatted transforms.

These results validate the data collection pipeline as a solid foundation for subsequent phases including 3DGS training and path planning development.

V DISCUSSION

5.1 Interpretation of Results

The experimental results demonstrate that the developed data collection pipeline successfully addresses the primary challenge of acquiring synchronized multi-modal datasets from ASVSim simulation. The 9× performance improvement achieved through rendering optimization is particularly significant, transforming an impractical process (55–60 seconds per frame) into an efficient workflow (6–7 seconds per frame).

This optimization was achieved through systematic analysis of the rendering pipeline, identifying that Lumen global illumination was the primary bottleneck. While Lumen produces photorealistic lighting effects, its computational cost for SceneCapture components is prohibitive for high-frequency data collection. The decision to disable Lumen and other post-processing effects represents a practical trade-off between visual fidelity and collection efficiency.

The 640×480 resolution, while lower than the initially planned 1280×720, proves sufficient for 3D Gaussian Splatting training. Literature suggests that 3DGS can achieve high-quality reconstruction at this resolution, particularly when combined with the accurate ground truth poses available from simulation (**kerbl20233d**). The reduced resolution may even be beneficial for training efficiency, allowing faster convergence during 3DGS optimization.

5.2 Technical Challenges and Solutions

Several technical challenges were encountered during development and addressed through iterative refinement:

5.2.1 Image Color Space Conversion

An initial issue with color accuracy (yellow/brown tint in water regions) was traced to RGB-BGR channel mismatch. CosysAirSim returns data in RGB order, while OpenCV's `imwrite` expects BGR. The fix involved explicit conversion using `cv2.cvtColor` with `COLOR_RGB2BGR`:

```
img = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)
```

This issue highlights the importance of understanding data format conventions when integrating multiple libraries.

5.2.2 Camera Position Optimization

Early configurations showed vessel structure (deck, mast) in camera views, which would contaminate 3DGS training with dynamic objects. Through systematic adjustment of camera position (X: 2.5→3.5m, Z: -1.8→-2.5m) and field of view (90→70°), the vessel was successfully excluded from the field of view.

5.2.3 CosysAirSim API Compatibility

Several API behavioral differences from standard AirSim required accommodation:

- `ImageType.DepthPlanar` uses value 1 (not 2)
- `simPause` causes scene freezing in v3.0.1
- `simGetImages` uses integer indices (not sensor names)
- Images are already upright (no `flipud` needed)

These differences were identified through experimentation and documented for future reference.

5.2.4 RPC Call Optimization

Initial implementations made separate `simGetImages` calls for each camera, creating RPC overhead. Consolidating all image requests into a single call significantly improved performance.

5.3 Limitations

Several limitations of the current work should be acknowledged:

5.3.1 Environment Fidelity

The LakeEnv environment, while useful for pipeline validation, lacks ice features essential for polar navigation research. Future work requires migration to polar-specific environments with icebergs, pack ice, and ice-covered water surfaces.

5.3.2 Trajectory Diversity

Current trajectories (circular, linear, random walk) provide basic coverage but may not capture all viewpoints optimal for 3DGS. More sophisticated trajectory planning considering view overlap and baseline constraints could improve reconstruction quality.

5.3.3 Segmentation Ground Truth

The pipeline skips real-time segmentation to improve efficiency. While segmentation can be generated offline, this requires additional processing steps. Integration of efficient real-time segmentation would streamline the workflow.

5.3.4 Coordinate System Conversion

The `build_transform_matrix` function for COLMAP format is currently incomplete, returning identity rotation matrices. Full implementation requires proper quaternion-to-rotation-matrix conversion and coordinate system alignment between ASVSim (X-forward, Y-right, Z-down) and COLMAP conventions.

5.3.5 Dynamic Elements

The current system assumes static environments. Real polar environments include dynamic ice floes, changing lighting conditions, and weather effects that are not yet incorporated.

5.4 Comparison with Related Work

Compared to existing marine simulation datasets (**bingham2019virtual**; **ebadi2020lakes**), this work provides:

Advantages:

- Multi-modal synchronization (RGB, depth, LiDAR, pose)
- Higher frame rates through rendering optimization
- Direct COLMAP format output
- Configurable collection parameters

Disadvantages:

- Lower resolution (640×480 vs. 1920×1080)
- Non-polar environment (LakeEnv vs. ice-covered)
- No semantic segmentation annotations

The trade-offs reflect prioritization of collection efficiency and pipeline completeness over maximum resolution and environment fidelity.

5.5 Implications for Polar Navigation

While the current implementation uses a lake environment, the methodology directly transfers to polar scenarios:

- Sensor configurations (camera, LiDAR) remain applicable
- Data formats and synchronization mechanisms are environment-agnostic
- Trajectory strategies can be adapted for ice navigation patterns
- The 3DGS reconstruction pipeline is independent of scene content

The key requirement for polar application is access to appropriate UE5 assets depicting ice formations, which are available through the Unreal Marketplace (Fab) (**fab2024arctic**).

5.6 Future Research Directions

Based on the limitations and opportunities identified, several directions for future research emerge:

5.6.1 Polar Environment Integration

Acquire and integrate polar-specific assets (icebergs, ice sheets, ice floes) into the simulation environment. This will enable collection of datasets directly relevant to polar navigation scenarios.

5.6.2 Perception Module Development

Integrate SAM3 and Depth Anything 3 for real-time ice detection and depth estimation. Fine-tune these models on collected datasets to improve domain-specific performance.

5.6.3 3DGS Training and Evaluation

Complete the 3DGS reconstruction pipeline by:

- Implementing proper pose transformation
- Training 3DGS models on collected datasets
- Evaluating reconstruction quality (PSNR, SSIM)
- Exploring multi-scale progressive 3DGS for polar scenes

5.6.4 Path Planning Implementation

Develop and integrate path planning algorithms:

- Global planners (A*, D* Lite) using 3DGS maps
- Local planners (VFH+, DWA) using LiDAR data
- Dynamic re-planning triggered by perception updates

5.6.5 Sim-to-Real Transfer

Investigate techniques for transferring algorithms trained in simulation to real-world polar conditions, including domain adaptation and sim-to-real validation strategies.

5.7 Contribution to the Field

This work contributes to marine autonomous navigation research by:

1. **Establishing best practices** for multi-modal data collection in ASVSim, including rendering optimization and sensor configuration
2. **Demonstrating feasibility** of efficient large-scale dataset generation from high-fidelity marine simulation
3. **Providing a reusable pipeline** that bridges simulation and 3D reconstruction workflows
4. **Documenting technical solutions** to common integration challenges (color spaces, API differences, coordinate systems)

The methodology and lessons learned are transferable to other simulation-based marine robotics research, beyond the specific polar navigation application.

VI CONCLUSION

6.1 Summary of Work

This thesis presented a comprehensive framework for polar route planning and 3D reconstruction using Unreal Engine and 3D Gaussian Splatting. The research successfully addressed the foundational challenge of multi-modal data collection from high-fidelity marine simulation, establishing a pipeline that bridges simulation environments and 3D reconstruction workflows.

The main achievements of this research are:

1. Simulation Platform Establishment: Successfully deployed and configured ASVSim on Unreal Engine 5 with multi-sensor setup including front-facing camera and top-mounted LiDAR. The configuration was optimized through systematic experimentation to balance visual quality with computational efficiency.

2. Rendering Optimization: Identified and resolved critical performance bottlenecks, achieving a 9× speed improvement (from 55–60 seconds to 6–7 seconds per frame) through disabling Lumen global illumination and reducing resolution to 640×480. This optimization transforms impractical data collection into an efficient workflow.

3. Multi-Modal Data Pipeline: Developed an automated collection system that captures synchronized RGB images, depth maps, LiDAR point clouds, and accurate ground truth poses. The pipeline includes robust error handling, retry mechanisms, and automatic validation.

4. Data Format Integration: Implemented conversion to COLMAP format, enabling direct use of collected data with standard 3D Gaussian Splatting training pipelines. The format preserves accurate camera intrinsics and extrinsics from simulation.

5. Experimental Validation: Demonstrated the system through successful collection of 200-frame datasets with 100% success rate, validating both the technical implementation and practical usability.

6.2 Key Findings

Several key findings emerged from this research:

- **Rendering configuration is critical:** The choice of rendering settings has dramatic impact on collection performance. Lumen GI, while producing beautiful visuals, is incompatible with high-frequency data acquisition.
- **Resolution can be traded for speed:** The 640×480 resolution, while modest by modern standards, provides sufficient quality for 3DGS reconstruction while enabling practical collection speeds.

- **API compatibility requires attention:** CosysAirSim exhibits several behavioral differences from standard AirSim that must be understood and accommodated for successful integration.
- **Ground truth poses are valuable:** Simulation provides perfect pose information, eliminating the need for Structure-from-Motion and enabling more efficient 3DGS training.

6.3 Contributions

This work makes the following contributions to the field of autonomous marine navigation:

1. **Methodological contribution:** A systematic approach to optimizing simulation-based data collection for 3D reconstruction, including identification of performance bottlenecks and practical solutions.
2. **Technical contribution:** An open, documented pipeline for multi-modal marine data collection that can be adapted and extended by other researchers.
3. **Empirical contribution:** Experimental validation demonstrating the feasibility of efficient large-scale dataset generation from high-fidelity marine simulation.

6.4 Limitations

The current implementation has several limitations that should be acknowledged:

- The environment (LakeEnv) lacks ice features essential for polar navigation
- Segmentation ground truth is generated offline rather than in real-time
- The coordinate transformation for COLMAP format requires completion
- Dynamic environmental effects are not yet incorporated

These limitations are recognized and planned as future work rather than deficiencies in the current contribution.

6.5 Future Work

Building upon the foundation established in this thesis, several directions for future research are identified:

Immediate extensions:

- Complete the COLMAP pose transformation implementation
- Integrate polar environment assets (icebergs, ice sheets)
- Implement real-time segmentation using SAM3
- Train and evaluate 3DGS models on collected datasets

Medium-term goals:

- Develop path planning algorithms using 3DGS reconstructions
- Integrate depth estimation (Depth Anything 3) for perception
- Implement dynamic obstacle handling
- Validate sim-to-real transfer capabilities

Long-term vision:

- Deploy the system on physical ASVs for real-world validation
- Extend to collaborative multi-vessel scenarios
- Incorporate weather and ice dynamics models
- Contribute to safe autonomous navigation in Arctic shipping lanes

6.6 Closing Remarks

As Arctic shipping routes become increasingly viable due to climate change, the need for intelligent autonomous navigation systems grows more urgent. This thesis establishes foundational infrastructure—simulation environment, data collection pipeline, and reconstruction workflow—that enables continued progress toward this goal.

The work demonstrates that high-fidelity simulation, when properly configured, can efficiently generate the diverse datasets needed for training modern perception and planning algorithms. The optimization strategies and technical solutions documented here will benefit other researchers working at the intersection of marine robotics and computer vision.

While significant challenges remain, particularly in transferring capabilities from simulation to real polar environments, the progress achieved provides a solid foundation for continued research. The dream of safe, autonomous navigation through Arctic waters—supporting sustainable shipping while protecting fragile polar ecosystems—is one step closer to realization.

A DATA COLLECTION SOURCE CODE

1.1 Main Collection Script (3-collect_dataset_v2.py)

The complete source code for the multi-modal data collection pipeline is provided below. This script implements the optimized data collection system described in Chapter III.

Listing 7: Collection Configuration Class

```
@dataclass
class CollectionConfig:
    """Collection configuration dataclass."""
    vessel_name: str = "Vessel1"
    front_cam_idx: int = 0
    lidar_name: str = "top_lidar"
    num_frames: int = 200
    move_seconds: float = 0.3
    thrust: float = 0.3
    angle: float = 0.6
    trajectory_mode: str = "circle"
    cam_w: int = 640
    cam_h: int = 480
    cam_fov: float = 90.0
    dataset_root: str = "dataset"
    max_retries: int = 3
    retry_delay: float = 1.0
    max_fail_ratio: float = 0.2
```

Listing 8: Camera Intrinsics Computation

```
def compute_intrinsics(w: int, h: int, fov_deg: float) -> Dict:
    """
    Compute camera intrinsics matrix K.
    Formula: fx = w / (2 * tan(FOV/2))
    """
    fov_rad = math.radians(fov_deg)
    fx = w / (2 * math.tan(fov_rad / 2))
    return {
        "fx": fx, "fy": fx,
        "cx": w / 2.0, "cy": h / 2.0,
        "width": w, "height": h,
```

```
        "fov_deg": fov_deg,
    }
```

Listing 9: Image Decoding with Color Space Conversion

```
def decode_image_safe(response, name: str = "") -> np.ndarray:
    """
    Safely decode images, handling RGB/RGBA formats
    and converting to BGR for OpenCV compatibility.
    """
    raw = np.frombuffer(response.image_data_uint8, dtype=np.uint8)
    h, w = response.height, response.width
    total = len(raw)

    expected_3ch = h * w * 3
    expected_4ch = h * w * 4

    if total == expected_4ch:
        # RGBA - convert to BGR
        img = raw.reshape(h, w, 4)[:, :, :3]
        img = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
    elif total == expected_3ch:
        # RGB - convert to BGR
        img = raw.reshape(h, w, 3)
        img = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
    else:
        # Unknown format - attempt recovery
        channels = max(1, total // (h * w))
        img = raw.reshape(h, w, channels)
        if channels > 3:
            img = img[:, :, :3]
        if len(img.shape) == 3 and img.shape[2] == 3:
            img = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
    return img
```

1.2 ASVSim Settings Configuration

The complete `settings.json` configuration for optimized data collection:

Listing 10: Optimized ASVSim Settings

```
{
  "SettingsVersion": 2.0,
  "SimMode": "Vessel",
  "PhysicsEngineName": "VesselEngine",
  "ViewMode": "SpringArmChase",
  "InitialInstanceSegmentation": true,
  "ClockSpeed": 1,
  "Wind": { "X": 0, "Y": 0, "Z": 0 },

  "PawnPaths": {
    "BlueResearchBoat": {
      "PawnBP": "Class'/AirSim/Blueprints/BP_VesselPawn.
         BP_VesselPawn_C'"
    }
  },

  "Vehicles": {
    "Vessel1": {
      "VehicleType": "MilliAmpere",
      "AutoCreate": true,
      "PawnPath": "BlueResearchBoat",
      "HydroDynamics": { "hydrodynamics_engine": "FossenCurrent" },
      "X": 0, "Y": 0, "Z": 0,
      "Sensors": {
        "top_lidar": {
          "SensorType": 6, "Enabled": true,
          "NumberOfLasers": 16,
          "PointsPerSecond": 300000,
          "RotationsPerSecond": 10,
          "Range": 100.0,
          "DrawDebugPoints": false,
          "DataFrame": "SensorLocalFrame",
          "X": 0.0, "Y": 0.0, "Z": -1.0
        },
        "front_camera": {
          "SensorType": 1, "Enabled": true,
          "Width": 640, "Height": 480,
```

```
          "X": 3.5, "Y": 0.0, "Z": -2.5,
          "Pitch": 0.0, "Roll": 0.0, "Yaw": 0.0
        }
      }
    }
  },

  "CameraDefaults": {
    "CaptureSettings": [
      {
        "ImageType": 0,
        "Width": 640, "Height": 480,
        "FOV_Degrees": 70,
        "LumenGIEnable": false,
        "LumenReflectionEnable": false,
        "MotionBlurAmount": 0,
        "ChromaticAberrationScale": 0.0,
        "AutoExposureBias": 0.0
      },
      {
        "ImageType": 1,
        "Width": 640, "Height": 480,
        "FOV_Degrees": 70,
        "LumenGIEnable": false,
        "LumenReflectionEnable": false,
        "MotionBlurAmount": 0
      },
      {
        "ImageType": 5,
        "Width": 640, "Height": 480,
        "FOV_Degrees": 70,
        "LumenGIEnable": false,
        "LumenReflectionEnable": false
      }
    ]
  }
}
```

1.3 Dataset Validation Script

The validation script checks dataset integrity:

Listing 11: Dataset Validation Function

```
def validate_dataset(dirs: Dict[str, Path],
                     expected_frames: int,
                     collected_frames: int) -> Dict:
    """Validate dataset completeness and quality."""
    results = {
        "valid": True,
        "issues": [],
        "stats": {},
    }

    # Check file counts
    rgb_files = sorted(list(dirs["rgb"].glob("*.png")))
    depth_files = sorted(list(dirs["depth"].glob("*.npy")))

    results["stats"]["rgb_count"] = len(rgb_files)
    results["stats"]["depth_count"] = len(depth_files)

    # Validate image quality
    if rgb_files:
        img = cv2.imread(str(rgb_files[0]))
        if img.shape != (480, 640, 3):
            results["valid"] = False
            results["issues"].append(f"Wrong dimensions: {img.shape}"
                )

    # Check pose file
    poses_path = dirs["base"] / "poses.json"
    if poses_path.exists():
        with open(poses_path, "r") as f:
            poses = json.load(f)
        if len(poses) != collected_frames:
            results["valid"] = False
            results["issues"].append("Pose count mismatch")

```

```
    return results
```